

Aprendizaje profundo aplicado a clasificación y regresión

Detección de fraude con tarjetas y estimación de precios de viviendas

Resumen ejecutivo

El proyecto compara algoritmos de machine learning y redes neuronales en dos problemas supervisados. El primer caso aborda la detección de transacciones fraudulentas en un conjunto fuertemente desbalanceado. El segundo analiza la estimación del precio de viviendas mediante modelos lineales, ensambles y una red neuronal de regresión. La comparación permite observar cómo cambian las métricas, el preprocesamiento y la arquitectura según el tipo de variable objetivo.

1. Objetivo y alcance

El objetivo es evaluar la capacidad de diferentes familias de modelos para resolver problemas de clasificación y regresión, incorporando redes neuronales densas como alternativa a métodos basados en árboles y modelos lineales. El análisis no se limita a seleccionar el valor más alto de una métrica: también considera el desbalance de clases, la preparación de variables, la separación entre entrenamiento y prueba y la interpretación de los errores.

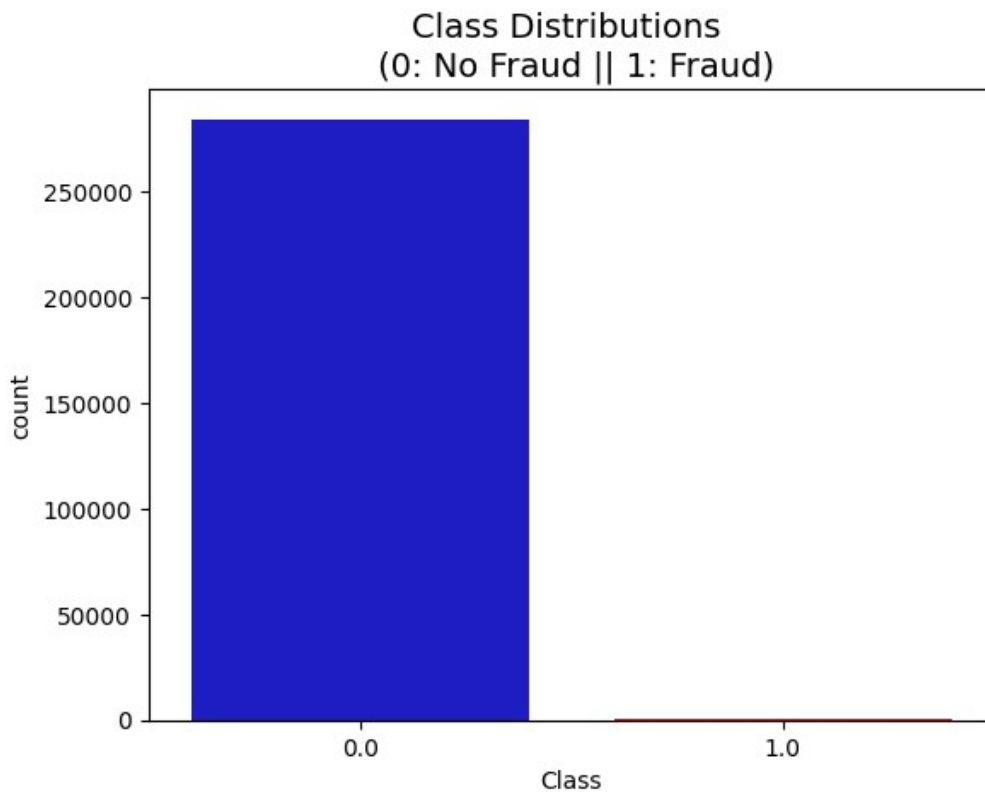
En clasificación, la variable objetivo identifica si una transacción corresponde o no a fraude. En regresión, la variable objetivo es el precio de una vivienda. En ambos casos se emplean datos estructurados y se comparan modelos con distintos niveles de complejidad.

2. Clasificación de transacciones fraudulentas

2.1 Conjunto de datos

La base contiene 284.807 transacciones realizadas con tarjetas de crédito y 31 variables. Las columnas V1 a V28 corresponden a variables transformadas, mientras que Time, Amount y Class completan la estructura. La clase 0 representa operaciones legítimas y la clase 1 identifica fraude.

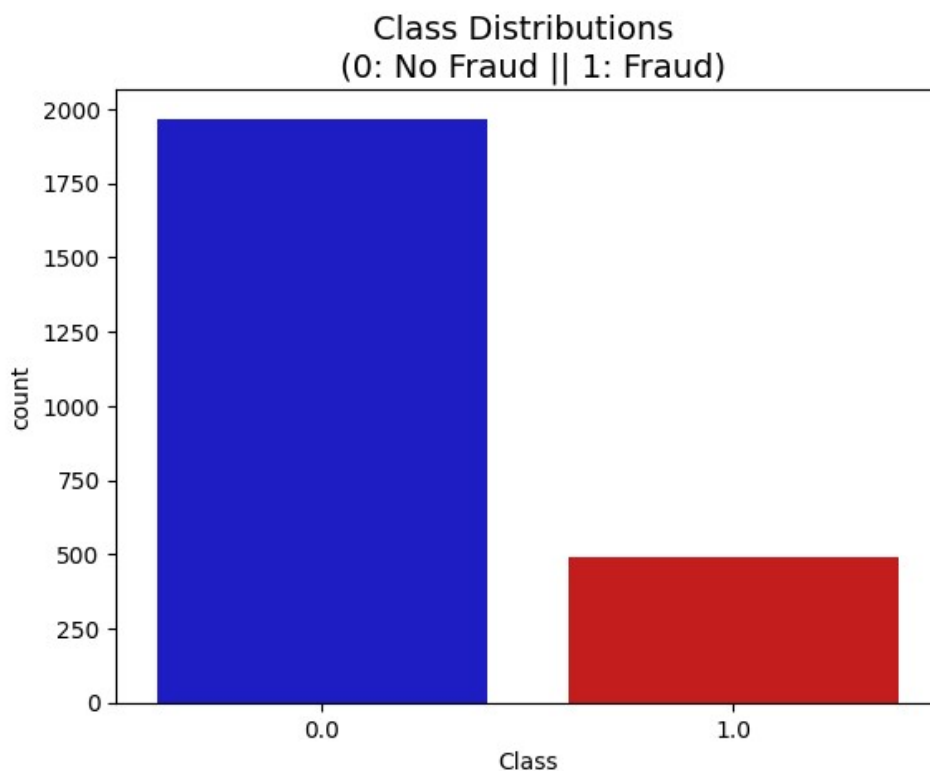
La distribución es extremadamente desigual: 284.315 registros pertenecen a la clase no fraude y 492 a fraude. Esto equivale aproximadamente a 99,83% y 0,17%, respectivamente. En este contexto, la accuracy aislada puede resultar engañosa, porque un modelo que prediga siempre la clase mayoritaria obtendría un valor muy alto sin detectar casos fraudulentos.



Distribución de clases en la base completa: 284.315 transacciones legítimas frente a 492 fraudes.

2.2 Preparación y muestra de modelado

1. Mezcla aleatoria de las observaciones para evitar que el orden del archivo condicione el entrenamiento.
2. Normalización min-max de todas las variables al intervalo [0, 1].
3. Separación de las clases y submuestreo de la clase mayoritaria.
4. Construcción de una muestra de 2.460 registros: 1.968 operaciones legítimas y 492 fraudes, de modo que la clase minoritaria represente 20%.
5. División estratificada en 50% para entrenamiento y 50% para prueba, utilizando random_state=42 en los modelos clásicos.



Muestra utilizada para comparar los clasificadores: 80% no fraude y 20% fraude.

2.3 Métricas de evaluación

La evaluación prioriza F1, precision y recall. Precision mide qué proporción de las alertas de fraude fueron correctas. Recall indica qué proporción de los fraudes reales fue detectada. F1 resume ambas medidas mediante su media armónica, por lo que resulta especialmente útil cuando las clases están desbalanceadas.

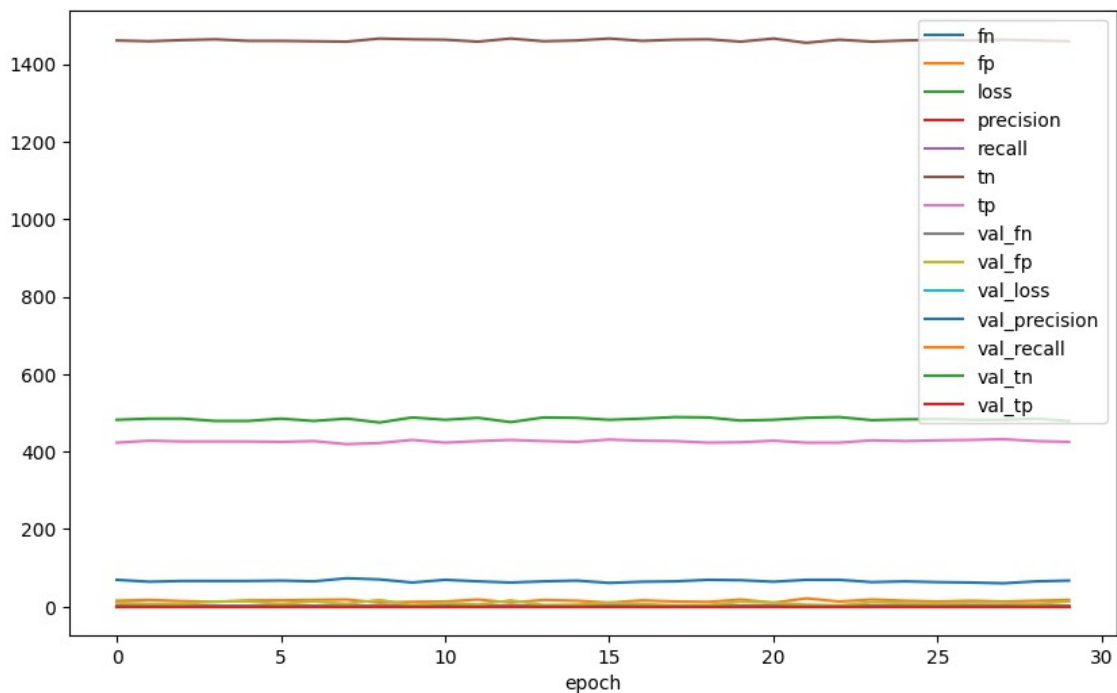
2.4 Modelos clásicos

- Decision Tree: genera reglas de decisión mediante particiones sucesivas del espacio de variables. Es interpretable, aunque puede presentar alta varianza.
- Random Forest: combina múltiples árboles entrenados sobre muestras y subconjuntos de variables. El ensamble reduce la inestabilidad de un árbol individual.
- Gradient Boosting: construye modelos de manera secuencial, concentrándose en los errores de las iteraciones anteriores.

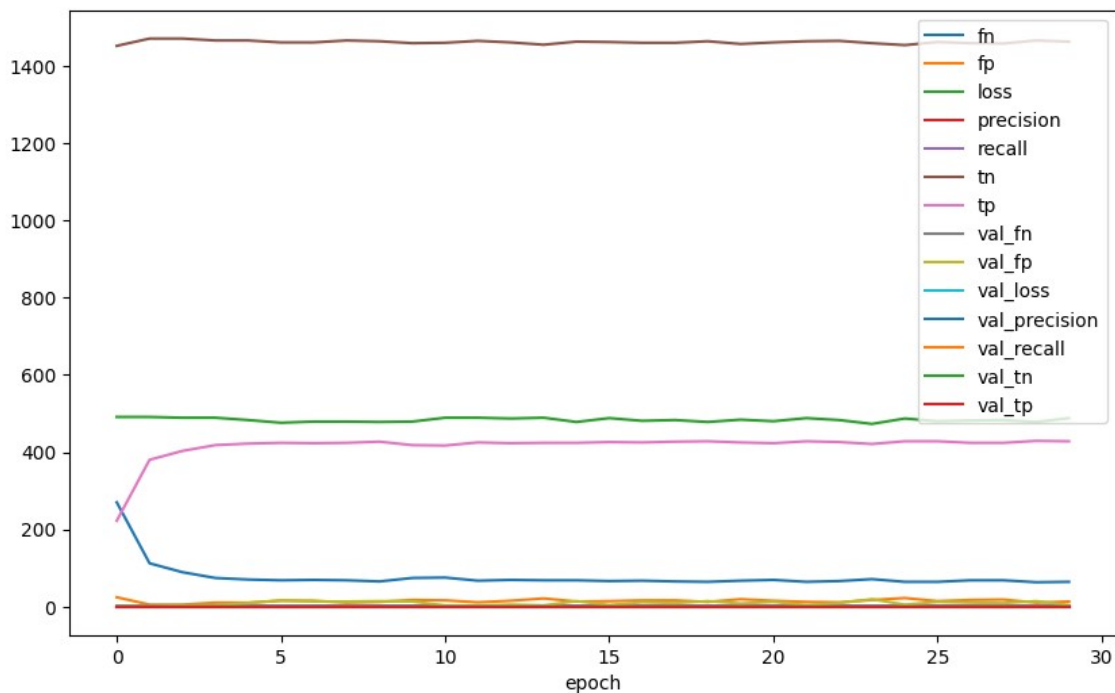
2.5 Redes neuronales densas

Se implementaron dos variantes de Keras. La arquitectura secuencial utiliza una entrada de 30 variables, capas densas de 256 y 128 unidades con activación lineal y una salida sigmoide para clasificación binaria. La versión funcional mantiene una entrada de 30 variables y capas densas conectadas mediante la API funcional. Ambas se compilan con el optimizador Adam y binary crossentropy.

El entrenamiento se realiza durante 30 épocas, con batch size 32 y validation_split=0,2. Además de precision y recall, se registran falsos negativos, falsos positivos, verdaderos negativos y verdaderos positivos, lo que permite analizar el costo de los errores de clasificación.



Evolución de las métricas durante el entrenamiento de la red Keras secuencial.



Evolución de las métricas durante el entrenamiento de la red Keras funcional.

2.6 Resultados de clasificación

Modelo	F1	Precision	Recall
Decision Tree	0.8649	0.8851	0.8455
Random Forest	0.9087	0.9766	0.8496
Gradient Boosting	0.9017	0.9505	0.8577
Keras secuencial	-	0.972	0.874
Keras funcional	-	0.9679	0.8322

Random Forest obtiene el F1 más alto entre los modelos clásicos (0,9087) y la mayor precision (0,9766), lo que indica una baja proporción de falsos positivos. Gradient Boosting presenta el recall más alto entre los modelos clásicos (0,8577), por lo que recupera una proporción ligeramente mayor de fraudes. Decision Tree queda por debajo en las tres métricas.

Las redes Keras alcanzan precision elevada, pero los resultados deben interpretarse junto con el procedimiento de validación y con el hecho de que el entrenamiento utiliza la muestra balanceada. En una implementación operativa sería necesario evaluar nuevamente sobre una distribución representativa de producción y seleccionar el umbral de decisión según el costo de falsos positivos y falsos negativos.

3. Regresión de precios de viviendas

3.1 Conjunto de datos y variable objetivo

La base boston_housing_esp.csv contiene 1.728 viviendas y 16 variables. La variable objetivo es precio. Las variables explicativas combinan atributos cuantitativos y cualitativos asociados con tamaño, antigüedad, terreno, equipamiento, servicios y localización.

- Dimensiones y antigüedad: metros_totales, metros_habitables y antigüedad.
- Valor y entorno: precio_terreno y universitarios.
- Distribución interior: dormitorios, baños, habitaciones y chimenea.
- Servicios: calefaccion, consumo_calefaccion, desagüe y aire_acondicionado.
- Atributos diferenciales: vistas_lago y nueva_construccion.

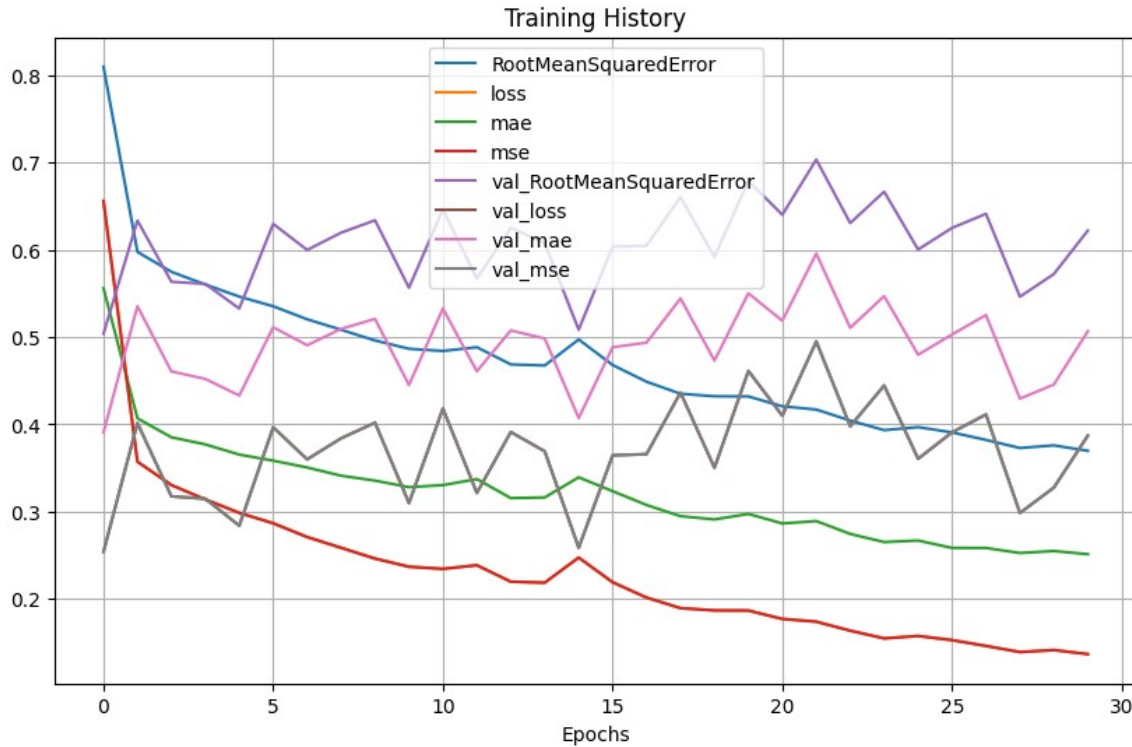
3.2 Preprocesamiento

1. Separación de la variable precio respecto de las variables predictoras.
2. Estandarización de variables numéricas mediante StandardScaler.
3. Codificación one-hot de variables categóricas, permitiendo categorías desconocidas en etapas posteriores.
4. Construcción de un pipeline de transformación para aplicar de forma consistente el preprocesamiento.
5. División 80% entrenamiento y 20% prueba con random_state=1234.

3.3 Modelos evaluados

Se comparan regresión lineal, Ridge, Extra Trees, LightGBM, XGBoost y Random Forest. Los ensambles capturan relaciones no lineales y efectos de interacción, mientras que Linear y Ridge funcionan como referencias interpretables. Las métricas utilizadas son R^2 , MAE y RMSE.

La red neuronal funcional utiliza una entrada de 24 variables procesadas, dos capas densas de 256 y 128 unidades con activación ReLU y una salida lineal. Se entrena durante 30 épocas con batch size 32, validation_split=0,2, optimizador Adam y pérdida MSE. Durante el ajuste se registran MAE, MSE y RMSE.



Historial de entrenamiento y validación de la red de regresión.

3.4 Resultados de regresión

Modelo	Train score	R^2	MAE	RMSE
LGBM	0.9209	0.645	0.4062	0.6329
Random Forest	0.9562	0.637	0.419	0.64
Extra Trees	1.0	0.6357	0.4206	0.6411
Linear	0.6642	0.6121	0.4493	0.6616
Ridge	0.6642	0.6119	0.4495	0.6618
XGBoost	0.9949	0.604	0.4243	0.6685
Keras funcional	-	-	0.2572	0.3877

Entre los modelos clásicos, LightGBM presenta el mayor R^2 de prueba (0,6450) y el menor RMSE (0,6329). Random Forest y Extra Trees muestran train scores muy altos frente a R^2 de prueba cercanos a 0,64, diferencia que debe observarse como posible señal de sobreajuste. Linear y Ridge obtienen resultados muy similares, lo que indica que la regularización aplicada no modifica de manera sustancial el desempeño en esta configuración. La red Keras reporta el menor MAE (0,2572) y RMSE (0,3877). Dado que no se informa R^2 para esta fila y que la escala de la variable objetivo puede haber sido transformada durante el proceso, la comparación debe realizarse verificando que todas las métricas estén calculadas sobre la misma escala y sobre el mismo conjunto de prueba.

4. Conclusiones integradas

- La selección de métricas debe responder al problema: F1, precision y recall son centrales en fraude; R^2 , MAE y RMSE lo son en regresión.
- Random Forest es el modelo clásico más equilibrado para detección de fraude en la comparación realizada.
- Gradient Boosting ofrece el recall clásico más alto y puede ser relevante cuando el costo de no detectar fraude es elevado.
- Los ensambles superan a los modelos lineales en la estimación de precios, aunque algunos muestran diferencias importantes entre entrenamiento y prueba.

- La red neuronal de regresión alcanza los menores errores reportados, pero requiere una validación homogénea para confirmar la comparación.
- El aprendizaje profundo aporta flexibilidad para aprender relaciones no lineales, aunque aumenta la necesidad de controlar el preprocesamiento, la validación y la reproducibilidad.

5. Limitaciones y mejoras posibles

- Validar los clasificadores sobre la distribución completa y no únicamente sobre la muestra con 20% de fraude.
- Incorporar curvas precision-recall, matriz de confusión y ajuste del umbral de decisión.
- Utilizar validación cruzada o particiones repetidas para reducir dependencia de una única división.
- Comparar los modelos de regresión sobre una escala común y agregar intervalos de error.
- Aplicar regularización, early stopping y búsqueda de hiperparámetros en las redes neuronales.
- Documentar semillas aleatorias y versiones de bibliotecas para facilitar la reproducción.

Fuentes de datos y herramientas

- **Credit Card Fraud Detection**: base de transacciones europeas con tarjeta, archivo incluido en data/.
- **SaratogaHouses**: conjunto de viviendas de Saratoga County, adaptado en boston_housing_esp.csv.
- **TensorFlow y Keras**: implementación de las redes neuronales densas.
- **Scikit-learn**: modelos clásicos, preprocesamiento y métricas.
- **Código y resultados del proyecto**: notebook, figuras y tablas incluidos en el repositorio.